

Adapter

idea - "a construct which adapts an existing interface X to conform to the required interface Y "

Patterns of structural design patterns

- Consider a scenario where we are currently using a custom email sender function, now what if instead we use a third party vendor for email sender; we have to make code changes at all the places to fit the vendor's code.

Code

we first create an abstract class to enforce method inheritance.

```
class SendNotif(ABC)
```

```
@abstractmethod
```

```
def send(self, msg, sub body):  
    pass
```

our custom notification class

```
class CustomNotif(SendNotification):
```

```
    def send(self, resp, subj, body):  
        print(f"Sending email to  
              {resp} with subject  
              {subj} and body  
              {body}")
```

our vendor based class

```
class SESNotif:
```

```
    def send_email(self, resp, subj, body):  
        print(f"SES email,  
              {resp},  
              {subj},  
              {body}")
```

Note that the
method name is
drbb

Now, we have an order class
that internally uses the email
class

we have hard
coded
the class ~~for~~ used.

class order:

```
def __init__(self, email_service: SendNotif):
```

```
    self._email_service = email_service.
```

```
def notify_order(self, resp, sub, body):
```

```
    self._email_service.send(resp, sub, body)
```

This will work for custom notification classes
as it has implemented the "send"
method required by Abstract class. But
what about "send_email" of SES Notif.

For that purpose, we make an adapter class.

```
class SESNotifAdapter(SendNotif):
```

```
def __init__(self, ses_source: SESNotif):  
    self._email_service = ses_source.
```

```
def send(self, resp, sub, body):
```

```
    self._email_service.send_email(resp, sub, body)
```

example

```
custom_notify = CustomNotify()
```

```
ses_notify = SESNotify()
```

```
order_custom = Order(custom_notify)
```

```
order_custom.notify_order()
```

```
"it"
```

```
"order custom notify",
```

```
"order placed"
```

```
)
```

if here we now have to use the
adapter class

```
ses_adapter = SESNotifyAdapter(ses_notify)
```

```
order_ses = Order(ses_adapter)
```

```
order_ses.notify_order()
```

```
"it"
```

```
"order ses notify",
```

```
"order placed"
```

```
)
```